INTRODUCTORY DATA STRUCTURES & ALGORITHMS

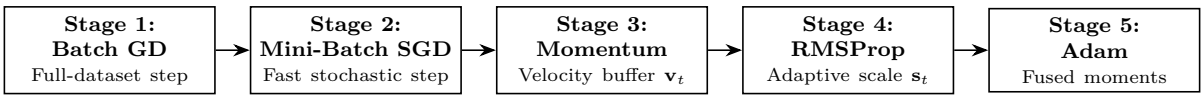
Official Guided Examination Papers (Problems 1–20)

STUDENT EDITION — STANDALONE EXAMINATION HANDOUTS

Examination Structure: The Progressive Algorithmic Ladder

Each examination paper in this collection investigates a foundational problem in computer science and scientific computing. Rather than presenting an optimized algorithm as a finished formula to memorize, each problem guides you through a **5-Stage Progressive Evolution Ladder**: you begin with the simplest baseline implementation, identify its computational or structural bottleneck on a concrete hand-worked instance, and apply a sequence of targeted adjustments to reach a state-of-the-art algorithm or data structure.

A classic illustration of this progression comes from continuous optimization:



Each standalone problem handout introduces all required definitions from first principles, illustrates the data structure mechanics visually, walks through concrete examples, and specifies the written and coding exercises for every stage.

Required Examination Deliverables (Per Assigned Problem)

For your assigned examination problem, you must submit three deliverables:

1. **Handwritten Step-by-Step Traces & Data Structure Explanations:**
   - **Manual Traces on Small Concrete Inputs:** Complete step-by-step hand calculations on the small arrays, matrices, trees, or graphs specified in the questions, showing all intermediate states and pointer/index updates.
   - **Data Structure Mechanics & Bottleneck Explanations:** Clear, plain-English explanations and simple diagrams showing how each stage stores its data in memory and what practical bottleneck of the previous stage it eliminates.
   - **Operation Counts & Big-O Summary Table:** Simple operation counts on concrete small inputs and a summary table comparing the time and space Big-O bounds of all five versions across the progression.
2. **Source Code Implementation & Empirical Benchmark Suite:**
   - Implement **all stages** of the algorithm/data structure progression (from the naive baseline through all intermediate adjustments to the final optimized version).
   - Provide automated unit tests verifying that all versions produce identical/valid outputs on shared test inputs.
   - Benchmark and plot **execution time** and **memory consumption** across increasing input sizes N, demonstrating empirically where each refinement overtakes its predecessor.
3. **Oral Defense Presentation Slides:**
   - A concise slide deck motivating the problem, walking through your hand-worked traces, explaining each step along the improvement ladder, and presenting your empirical runtime and memory plots.

PROBLEM 11: DISJOINT-SET UNION (UNION-FIND) & PERCOLATION

Dynamic Connectivity: From BFS Flood-Fill to Union by Rank, Path Compression & Percolation

Core Data Structures & Concepts: Component-ID Arrays, Parent-Pointer Up-Trees, Rank Bounds, Virtual Boundary Nodes

Algorithmic Progression: V1 (Graph BFS/DFS) → V2 (Quick-Find Array) → V3 (Naive Up-Trees) → V4 (Union by Rank) → V5 (Path Compression + Virtual Nodes)

Why Dynamic Connectivity Is One of Computer Science's Greatest Success Stories

Imagine you are managing a network of n elements (numbered $0, 1, \ldots, n-1$) that start completely disconnected. Over time, links arrive one by one in a live stream, and you must support two fundamental operations at blazing speed:

- `Union(u, v)`: Add a connection between u and v, merging their connected groups (sets) into one.
- `Connected(u, v)` (via `Find(u) == Find(v)`): Answer immediately whether u and v belong to the same connected group (even if only through a long chain of intermediate links).

This **Disjoint-Set Union (DSU)** or **Union-Find** data structure is the engine behind:

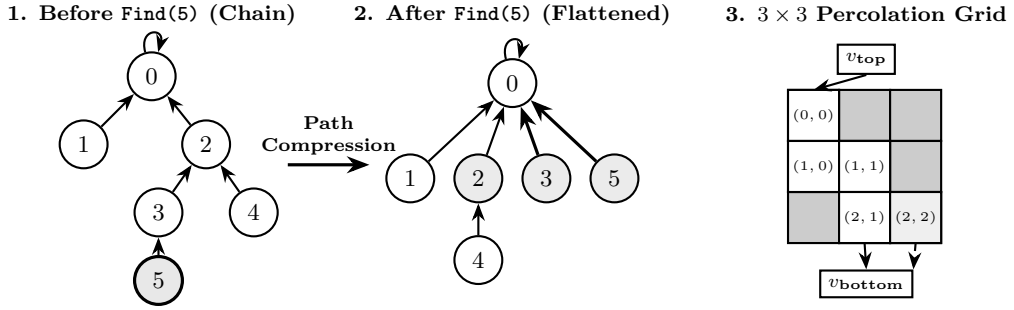
1. **Kruskal's Minimum Spanning Tree (MST) Algorithm:** Building the cheapest fiber-optic or road network connecting n cities by greedily adding the lightest edge that does not create a cycle (`Find(u) != Find(v)`).
2. **Image Processing & Medical Scans:** Labeling connected regions of pixels or 3D voxels in MRI scans.
3. **Physical Percolation & Phase Transitions:** Modeling how oil seeps through porous rock, how forest fires jump across trees, or how an $L \times L$ insulating grid suddenly conducts electricity the instant the fraction of open sites crosses a sharp mathematical threshold $p_c \approx 0.592746$.

As you will discover in this problem, evolving from a naive graph search to **Union by Rank with Path Compression** reduces the cost per operation from $\Theta(n)$ down to $O(\alpha(n))$—where $\alpha(n)$ grows so mind-bogglingly slowly that $\alpha(n) \leq 4$ for every input size n smaller than the number of atoms in the observable universe!

Visual Primer: Parent-Pointer Up-Trees, Path Compression & Virtual Grid Nodes

Instead of storing heavy pointers or adjacency lists, a Disjoint-Set forest represents *all* sets inside a **single flat integer array** `parent[0..n-1]`. Each element x points upward to its parent `parent[x]`. If `parent[r] == r`, then r is the **root** (canonical representative) of its entire tree!

- **To check if u and v are in the same set:** Follow `parent` pointers up from u to its root $r_u = \texttt{Find}(u)$, and from v to its root $r_v = \texttt{Find}(v)$. They are connected if and only if $r_u == r_v$.
- **To merge two sets (`Union(u, v)`):** Find their roots r_u and r_v. If $r_u \neq r_v$, simply make one root point to the other: `parent[r_u] = r_v`!



Look at the middle diagram: while walking up $5 \to 3 \to 2 \to 0$ during `Find(5)`, we already discovered that 0 is the root of 5, 3, and 2. **Path Compression** rewires `parent[5] = 0` and `parent[3] = 0` on the spot so future lookups on 5 or 3 finish in 1 hop!

The 5-Stage Dynamic Connectivity & Union-Find Evolution Ladder

Suppose we have n elements $\{0, 1, \ldots, n-1\}$ and process a stream of m operations (`Union` and `Connected`):

- **Stage 1 (V1): Explicit Adjacency-List Graph + Full BFS/DFS Flood-Fill per Query**
  - *Idea:* Store an undirected graph adjacency list `adj[0..n-1]`. `Union(u, v)` appends edge (u, v) in $O(1)$ time.
  - *Bottleneck:* Every `Connected(u, v)` query must run a fresh Breadth-First Search (BFS) from u to see if v is reachable, costing $\Theta(n + e)$ time *per query*! Across m queries, total runtime explodes to $\Theta(m(n + e))$.
- **Stage 2 (V2): Quick-Find Component-ID Array (`id[0..n-1]`)**
  - *Data Structure Upgrade:* Store a flat array `id[i]` (initialized to `id[i] = i`) holding the component number of element i. Now `Find(x)` simply returns `id[x]` in $\Theta(1)$ time!
  - *Bottleneck:* To execute `Union(u, v)`, if `old_id = id[u]` and `new_id = id[v]` differ, we must scan the **entire array** (`for i in range(n): if id[i] == old_id: id[i] = new_id`) to update every team member's badge. Thus every `Union` costs $\Theta(n)$, and $n - 1$ unions take $\Theta(n^2)$ time!
- **Stage 3 (V3): Naive Parent-Pointer Up-Trees (`parent[0..n-1]`)**
  - *Data Structure Upgrade:* Store `parent[i]` (initially `parent[i] = i`). `Find(x)` follows parent links up to the root (`while x != parent[x]: x = parent[x]`), and `Union(u, v)` links the two roots via a single assignment: `parent[Find(u)] = Find(v)`.
  - *Bottleneck:* If we repeatedly attach a tall tree under a single new node (e.g., `Union(0,1)`, `Union(1,2)`, ..., `Union(n-2,n-1)`), the tree degenerates into a stick of height $n - 1$, making `Find` take $\Theta(n)$ hops!
- **Stage 4 (V4): Weighted Union by Rank (or Size)**
  - *Algorithmic Fix:* Never attach a tall tree under a short tree! Maintain an array `rank[0..n-1]` (initialized to 0) recording an upper bound on each root's subtree height:

```
def union_by_rank(u, v):
    ru, rv = find(u), find(v)
    if ru == rv: return False
    if rank[ru] < rank[rv]:
        parent[ru] = rv          # shorter root attaches under taller root
    elif rank[ru] > rank[rv]:
        parent[rv] = ru
    else:
        parent[rv] = ru          # equal height: tie-break & increment rank
            by +1
        rank[ru] += 1
    return True
```

  - *Guarantee:* Because a tree's height only grows by +1 when it merges with another tree of *at least equal size*, every tree of rank k contains at least 2^k nodes. Thus tree height never exceeds $\lfloor \log_2 n \rfloor$, guaranteeing $O(\log n)$ worst-case time per `Find` and `Union`!
- **Stage 5 (V5): Path Compression + Virtual Boundary Nodes for Percolation**
  - *Algorithmic Fix:* Upgrade `find(x)` to flatten every node along the search path directly onto the root:

```
def find_compressed(x):
    root = x
    while root != parent[root]:
        root = parent[root]
    while x != root:             # Second pass: rewire every node directly
        to root!
```

```
        nxt = parent[x]
        parent[x] = root
        x = nxt
    return root
```

– *Percolation Virtual Node Trick:* On an $L \times L$ grid (with $N = L^2$ cells indexed $0 \dots L^2 - 1$), testing whether any open top-row cell connects to any open bottom-row cell would naively take $L \times L = L^2$ calls to `Connected`. By adding two extra elements $v_{\text{top}} = L^2$ (unioned with every open top-row cell) and $v_{\text{bottom}} = L^2 + 1$ (unioned with every open bottom-row cell), testing whether the entire grid percolates reduces to a **single call** `Connected(v_top, v_bottom)` in $O(\alpha(L^2))$ time!

Part A: Hand-Worked Traces Across the DSU Ladder & Kruskal's MST

Q1. Tracing V2 vs. V3 vs. V4 on a 6-Element Chain: Start with $n = 6$ isolated elements $\{0, 1, 2, 3, 4, 5\}$ and execute the 5 operations in order:

`Union(0, 1)`, `Union(1, 2)`, `Union(2, 3)`, `Union(3, 4)`, `Union(4, 5)`.

- **V2 (Quick-Find `id[]`):** Assume `Union(u, v)` changes all entries equal to `id[u]` into `id[v]`. Write the array `id[0..5]` after each of the 5 unions, and count the total number of array entries overwritten.
- **V3 (Naive Up-Tree `parent[]`):** Assume `Union(u, v)` sets `parent[Find(u)] = Find(v)` without rank checks. Write the final `parent[0..5]` array after all 5 unions. What is the height of the tree, and how many pointer hops does `Find(0)` require?
- **V4 (Union by Rank):** Using the `union_by_rank` code above (where equal-rank ties set `parent[rv] = ru` and increment `rank[ru]`), write the final `parent[0..5]` and `rank[0..5]` arrays after all 5 unions. What is the tree height now?

Q2. Building a Height-3 Binomial Tree & Tracing Path Compression (Stages 4 & 5): Starting with $n = 8$ elements $\{0, 1, \dots, 7\}$ under **Union by Rank alone (V4)**, execute:

- Round 1 (pairs): `Union(0,1)`, `Union(2,3)`, `Union(4,5)`, `Union(6,7)`.
- Round 2 (4-tuples): `Union(0,2)`, `Union(4,6)`.
- Round 3 (all 8): `Union(0,4)`.

Draw the resulting 8-node up-tree and write down `parent[0..7]` and `rank[0..7]`, verifying that node 7 sits at depth 3 ($7 \to 6 \to 4 \to 0$). Next, execute a **single call** to `find_compressed(7)` (Stage 5) and write the updated `parent[0..7]` array and the new depth of every node.

Q3. Kruskal's Minimum Spanning Tree (MST) Hand Trace: Consider a connected undirected graph $G = (V, E)$ on 7 vertices $\{A, B, C, D, E, F, G\}$ with 11 weighted edges:

$$(A, B, 7), \quad (A, D, 5), \quad (B, C, 8), \quad (B, D, 9), \quad (B, E, 7), \quad (C, E, 5),$$
$$(D, E, 15), \quad (D, F, 6), \quad (E, F, 8), \quad (E, G, 9), \quad (F, G, 11).$$

Sort all 11 edges in non-decreasing order of weight (breaking ties alphabetically by endpoints, e.g., $(A, D, 5)$ before $(C, E, 5)$, and $(A, B, 7)$ before $(B, E, 7)$). Trace Kruskal's algorithm step by step using Union-Find: for each edge (u, v, w), state `Find`(u), `Find`(v), whether the edge is **Accepted** (merged via `Union`) or **Rejected** (would form a cycle because `Find`$(u) ==$ `Find`(v)), and report the final 6 MST edges and their total weight.

Part B: The Percolation Backwash Bug & 5-Stage Complexity Table

Q4. Diagnosing & Fixing the Percolation "Backwash Bug" (Stage 5): Look at the 3×3 grid on the right side of the Visual Primer diagram above, where cells $(0, 0), (1, 0), (1, 1), (2, 1)$ form an open percolating path from the top row to the bottom row, and cell $(2, 2)$ at the bottom-right is *also* open, while $(0, 2)$ and $(1, 2)$ above it are blocked.

- Suppose we want to shade every **"full" cell** (an open cell that water pouring in from the top row $r = 0$ can actually reach) by checking `Connected(v_top, cell)`. Trace which cells belong to the same set as v_{top} when *both* v_{top} and v_{bottom} are wired into the same Union-Find structure. Why does `Connected(v_top, (2,2))` return `True` even though water from the top cannot physically reach $(2, 2)$?
- Explain how maintaining **two** Union-Find instances—`uf_perc` (with both v_{top} and v_{bottom} for testing `percolates()`) and `uf_full` (with v_{top} only, omitting v_{bottom}, for testing `is_full(r, c)`)—completely eliminates the Backwash Bug in $O(\alpha(L^2))$ time per step.

Q5. 5-Stage Complexity Comparison Table: Create a summary table comparing **V1 (Graph BFS)**, **V2 (Quick-Find Array)**, **V3 (Naive Up-Trees)**, **V4 (Union by Rank)**, and **V5 (Union by Rank + Path Compression)** across: (a) time per `Union`, (b) time per `Find`/`Connected`, (c) total time to process $n - 1$ unions + n connectivity queries on n elements, and (d) auxiliary space complexity.

Part C: Implementation & Empirical Benchmarking Lab (All 5 Versions)

Implement all five connectivity engines in Python and run two experiments:

1. **Experiment 1 (Adversarial Chain Scaling & Pointer-Hop Counter):** For $n \in \{500, 1000, 2000, 5000\}$, build a worst-case chain of $n - 1$ unions followed by n queries. Measure wall-clock runtime and average parent-pointer hops per `Find` across V1, V2, V3, V4, and V5. Verify that V4 caps depth at $\leq \log_2 n$ while V5 drops average hops per query to < 1.5.
2. **Experiment 2 (Monte Carlo Percolation Phase Transition on an $L \times L$ Lattice):** Using your V5 Union-Find (with the dual-UF backwash fix), simulate site percolation on an $L \times L$ grid for $L \in \{40, 80, 160\}$. Sweep open probability $p \in [0.45, 0.75]$ in steps of 0.01 (running 100 random trials per p) and plot the empirical percolation probability $\hat{\mathbb{P}}(\text{percolates} \mid p)$ versus p, verifying the sharp sigmoid phase transition at $p_c \approx 0.593$.